

PROJECT_CONTEXT.md

Judicial Case Management System (JCMS) — AI-Powered — Project Context Brief

Purpose of this document: a complete, self-contained snapshot of everything planned/designed so far, intended to be uploaded to a new AI chat session to resume work with zero lost context.

1. Executive Summary & Vision

Project Name: Judicial Case Management System (JCMS)

Core Purpose: A modern, production-ready, AI-powered web platform that streamlines court operations — case filings, hearing schedule optimization, emergency petition handling, and courtroom allocation — across four distinct user roles (Judge, Advocate, Registrar, Public).

Target Audience / Portfolio Goal: This is a flagship software engineering portfolio project, built by a **2-developer team**, intended to demonstrate to tech recruiters: full-stack architecture depth, algorithmic thinking (priority scheduling), real-time systems, applied AI/LLM integration (not just CRUD), and production/cloud-readiness (CI/CD, IaC, observability).

Differentiating pillars:

- Weighted, explainable priority-scheduling algorithm with emergency override workflow
 - Hash-chained, tamper-evident digital audit log
 - Real-time WebSocket-driven dockets
 - AI/LLM layer: petition extraction, AI-assisted priority urgency scoring, RAG-based case summarization, AI stenographer/draft-order generation — all human-reviewed, never auto-authoritative
-

2. Architecture & Tech Stack Decisions

Layer	Decision
Frontend	React (Vite) + TypeScript + TailwindCSS + shadcn/ui
State/data	TanStack Query + Zustand
Backend	Node.js + NestJS (TypeScript) or Spring Boot (Java) — team to finalize based on which resume signal they want; NestJS assumed as default in examples below
Database	PostgreSQL (+ <code>pgvector</code>) extension for embeddings)
ORM	Prisma (Node) or JPA/Hibernate (Java)
Auth	JWT (access + refresh) via Keycloak or Auth0, or Passport.js + argon2/bcrypt self-rolled
Caching	Redis (cache-aside for cause lists, sessions, rate limiting)
Queue	Redis Streams / BullMQ (or RabbitMQ for a more enterprise signal)
Real-time	WebSockets via Socket.IO
File storage	S3-compatible object storage (AWS S3 prod / MinIO local dev)
Search (public lookup)	PostgreSQL full-text search (MVP) → Elasticsearch/OpenSearch (Phase 3 scale-out)
AI/LLM — extraction & generation	GPT-4o/4.1 or Gemini 1.5/2.x (structured JSON outputs); Llama 3.1/3.3 via Ollama/vLLM as a self-hosted, provider-agnostic option
AI/LLM — embeddings	<code>text-embedding-3-large</code> (OpenAI) / <code>text-embedding-004</code> (Gemini) / <code>bge-large-en-v1.5</code> (open-source local)
AI/LLM — OCR	Tesseract OCR (open-source) or Google Document AI / AWS Textract
AI/LLM — speech-to-text	Whisper API or self-hosted <code>faster-whisper</code> + <code>pyannote.audio</code> for diarization
Vector DB	PGVector inside existing Postgres (chosen over Pinecone/ChromaDB for simplicity + transactional consistency; Pinecone documented as the future scale-out path)
Orchestration framework	LangChain or LlamaIndex (pick one, go deep)

Layer	Decision
Infra	Docker Compose (dev) → Kubernetes/ECS (prod) + Terraform (IaC)
CI/CD	GitHub Actions
Observability	Prometheus + Grafana, Sentry
Repo structure	Monorepo: <code>apps/api</code> , <code>apps/web</code> , <code>packages/shared-types</code>

3. Team Breakdown & Vertical Ownership

Split is by **full-stack vertical ownership** (not frontend/backend split) so each dev has an independently demoable slice and merge conflicts stay low.

Dev A — "Core Ops"

- **Case lifecycle backend:** Case/Hearing/Party CRUD, status state machine (Filed → Verification → Listed → Heard → Disposed)
- **Weighted Priority Scheduling Engine:** deterministic scoring (case age, statutory urgency class, emergency flag, vulnerability flags) backed by a **Redis sorted-set queue** for $O(\log N)$ reordering
- **Courtroom Load Balancer:** courtroom/time-slot allocation, conflict detection (double-booking, judge unavailability)
- **Judge Dashboard:** docket view, disposition recording, courtroom management, bench notes
- **Registrar Portal:** filing verification, queue management console, **emergency override workflow** (state machine: Filed(Emergency) → Registrar Triage → Judge Acceptance → Slot Injection → Notification fan-out)
- **Advocate Portal (case-side):** e-filing, case tracking, hearing calendar
- **Digital Audit Log:** hash-chained, append-only, tamper-evident; verification endpoint
- **Infra ownership (overall):** Docker Compose, Kubernetes/Terraform, CI/CD pipeline, load testing (k6), security hardening pass

Dev B — "Intelligence & Access"

- **Auth & RBAC:** JWT auth, role guards, Keycloak/Auth0 setup — exposes the `@Roles()` guard Dev A's routes consume
- **Document Vault:** S3/MinIO upload, versioning, checksum verification, virus-scan job (BullMQ)
- **Public/Litigant Portal:** public cause list, case-status lookup, order/judgment access
- **WebSocket Real-Time Layer:** Socket.IO infra, room management (`court:{id}` rooms), event contract (`queue:updated`, `hearing:scheduled`, `emergency:alert`)

- **AI/LLM & RAG Integration (full pipeline):**
 - Petition categorization & structured data extraction (OCR + LLM structured output)
 - AI priority urgency classifier (outputs a bounded suggestion into Dev A's scoring engine — never sets the score directly)
 - RAG-based case summarization ("Summarize File": chunking → PGVector embeddings → retrieval → cited generation)
 - AI stenographer: speech-to-text + diarization → structured draft court order (judge-reviewed only, never auto-published)
 - **Eval/quality:** labeled test set for extraction accuracy, prompt versioning, cost/latency tiering (cheap model first pass, escalate to larger model for ambiguous cases)
-

4. Data Models & API Contracts

4.1 Core Schema (Postgres)

```
User (id, name, email, password_hash,  
role[Judge|Advocate|Registrar|Admin|Public], bar_council_id?, created_at)  
  
Court (id, name, location, jurisdiction_type)  
  
Courtroom (id, court_id FK, room_number, capacity)  
  
Judge (id, user_id FK, court_id FK, designation, is_active)  
  
Case (id, case_number [unique, indexed], case_type, filing_advocate_id FK-  
>User,  
      status[Filed|Verification|Returned|Listed|Heard|Disposed],  
      priority_score (computed, indexed), is_emergency (bool),  
      filed_at, court_id FK, assigned_judge_id FK->Judge nullable)  
  
Party (id, case_id FK, name, role[Petitioner|Respondent], contact_info)  
  
Hearing (id, case_id FK, courtroom_id FK, judge_id FK, scheduled_at,  
          status[Scheduled|Completed|Adjourned|Cancelled], order_summary,  
          next_hearing_date)  
  
Document (id, case_id FK, uploaded_by FK->User,  
           doc_type[Petition|Evidence|Order|Affidavit],  
           storage_url, version, checksum, virus_scan_status, uploaded_at)  
  
PriorityFactor (id, case_id FK,  
               factor_type[Age|StatutoryUrgency|EmergencyFlag|VulnerabilityFlag],  
               weight, computed_score, computed_at)  
  
AuditLog (id, entity_type, entity_id, action, actor_id FK->User, before_state  
JSONB,  
          after_state JSONB, reason, prev_hash, hash, created_at)  
  
Notification (id, user_id FK, case_id FK, type, channel[Email|SMS|Push],  
             sent_at, read_at)
```

4.2 AI/Vector Extension to Schema (Dev B)

```
sql
```

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
CREATE TABLE document_chunk (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  document_id UUID REFERENCES "Document"(id),  
  case_id UUID REFERENCES "Case"(id),  
  chunk_text TEXT NOT NULL,  
  page_number INT,  
  embedding VECTOR(1536),  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

```
CREATE INDEX ON document_chunk USING ivfflat (embedding vector_cosine_ops) WITH
```

4.3 Key API Endpoints

Auth (Dev B)

```
POST    /api/auth/login
POST    /api/auth/refresh
POST    /api/auth/logout
```

Cases (Dev A)

```
POST    /api/cases
GET     /api/cases/:id
GET     /api/cases?status=&court=&q=
PATCH  /api/cases/:id/status
POST    /api/cases/:id/emergency-flag
GET     /api/cases/:id/audit-trail
```

Priority Queue (Dev A, consumes Dev B's AI suggestion)

```
GET     /api/courts/:courtId/queue
POST    /api/courts/:courtId/queue/reorder
POST    /api/scheduler/recompute
```

Hearings / Courtroom Allocation (Dev A)

```
POST    /api/hearings
PATCH  /api/hearings/:id
GET     /api/courtrooms/:id/availability
```

Documents (Dev B)

```
POST    /api/cases/:id/documents
GET     /api/documents/:id/download
```

AI Features (Dev B)

```
POST    /api/ai/petitions/:id/extract      -> structured JSON extraction
POST    /api/ai/cases/:id/priority-suggestion -> { urgency_class, rationale,
suggested_weight_delta }
POST    /api/ai/cases/:id/summarize        -> RAG summary with citations
POST    /api/ai/hearings/:id/draft-order   -> draft order from transcript
(review-only)
```

Public (Dev B)

```
GET     /api/public/cause-list?court=&date=
GET     /api/public/case-status?caseNumber=
```

Realtime (Dev B – Socket.IO namespace, Dev A emits into it)

```
/ws/court/:courtId -> queue:updated, hearing:scheduled, emergency:alert
```

4.4 Critical Integration Contract — Priority Score Interface

This is the single most important seam between the two devs' work. Dev A owns and runs the deterministic weighted-scoring engine; Dev B's AI classifier must conform to this output shape and **never writes the final score directly**:

```
ts

interface AIPrioritySuggestion {
  urgency_class: "high" | "medium" | "low";
  rationale: string;
  flagged_factors: string[]; // e.g. ["irreparable harm", "custodial dete
  suggested_weight_delta: number; // bounded, e.g. -15 to +15
}
```

Other agreed contracts (all live in `CONTRACTS.md` in repo root): shared TypeScript types package, auth guard/decorator shape, WebSocket event payload shapes, `DocumentSummary` type for case-document linking.

5. AI & Feature Pipeline Specifications

5.1 AI Petition Categorization & Data Extraction

- Input: filed petition PDF (digital or scanned)
- OCR fallback: Tesseract / Document AI / Textract for scanned docs
- Extraction: LLM with structured/JSON-schema output → Case Title, Party names, Case Type, key dates, relief sought
- Validation: deterministic regex/fuzzy-match checks before DB write
- **Human-in-the-loop**: registrar confirms/edits pre-filled form before commit — never auto-committed

5.2 Intelligent Priority Score Estimator

- LLM reads petition narrative → outputs `AIPrioritySuggestion` (see 4.4)
- Feeds as a **bounded delta** into Dev A's deterministic scoring engine — algorithm stays fully auditable
- Cost optimization: cheap/small model as first-pass classifier, escalate to larger model only for ambiguous cases
- All suggestions + human-adjusted final scores logged for future eval/fine-tuning

5.3 Smart Case Summarization (RAG)

- Chunk all case documents (~500–1000 tokens, overlap) with metadata (`case_id`, `doc_type`, `date`, `page`)
- Embed via OpenAI/Gemini/or local `bge-large-en` → store in **PGVector**

- Retrieval: filter by `case_id` first, then rank by cosine similarity (two-stage filter-then-rank)
- Generation: long-context model produces structured summary (Parties, Timeline, Key Evidence, Prior Orders, Open Issues)
- **Citations required:** each summary claim links back to its source chunk/page

5.4 AI Stenographer / Transcript Summarizer

- Speech-to-text: Whisper (API or self-hosted) + `pyannote.audio` diarization
- LLM structures diarized transcript into a draft order (case number, appearances, submissions, directions, next date)
- Output explicitly labeled "**AI-generated draft — pending judicial review**"; only becomes official after judge edits/approves

5.5 Real-Time WebSocket Docket Updates

- Socket.IO, rooms scoped per court (`court:{courtId}`)
- Events: `queue:updated` (registrar reorders → judge dashboard patches live), `hearing:scheduled`, `emergency:alert` (urgent petition routed to bench instantly)
- Dev B owns the Socket.IO server/event contract; Dev A's domain logic emits into it on state changes

6. Current Status & Next Steps

Where we left off

All planning/design phases are complete:

1. ☒ Core feature scope, architecture, DB schema, API design, phase roadmap (Phase 1 MVP → Phase 3 cloud-ready) — established
2. ☒ AI/LLM integration blueprint (extraction, priority estimator, RAG summarization, AI stenographer, vector DB choice) — established
3. ☒ 2-developer vertical work split (Dev A: Core Ops, Dev B: Intelligence & Access) with week-by-week phase mapping — established
4. ☒ This context brief consolidates all of the above into one resumable document

No code has been written yet — the project is fully in the planning/architecture stage.

Immediate next tasks (pick up here in the next session)

1. **Finalize backend language choice** (NestJS/TypeScript vs Spring Boot/Java) — currently undecided, examples assume NestJS.
2. **Scaffold the monorepo:** `apps/api`, `apps/web`, `packages/shared-types`, Docker Compose (Postgres + Redis + MinIO), base CI pipeline.
3. **Write** `CONTRACTS.md` formalizing the 5 integration contracts (shared types, priority-score interface, auth guard shape, WebSocket event shapes, document-summary

type).

4. **Implement Phase 1, Week 1 tasks in parallel:**

- Dev A: DB schema migration + Case/Hearing CRUD
- Dev B: JWT auth + role guards + User model

5. **Generate first concrete code artifacts** (natural next prompts): the Prisma/JPA schema file, the NestJS module structure, the priority-scoring algorithm pseudocode/implementation, or the extraction prompt + JSON schema for petition categorization.